Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

Name: Devyani Kamat EXPERIMENT NO. 5 SAP : 60017240102

AIM:

Establish a database layer using Prisma ORM with PostgreSQL/MySQL to wire relationship logic and handle full transactional operations.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Container Engine:** Docker Engine & Docker Compose (or Docker Desktop)
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Containerized Database Infrastructure (Docker Compose):** Docker enables isolated, reproducible development environments. Using Docker Compose, an isolated PostgreSQL database engine runs with persistent named storage volumes (postgres_data), ensuring consistent local infrastructure without requiring manual database installations on host operating systems.
- **Prisma 7 Architecture & Programmatic Configuration** (prisma.config.ts)**:** In Prisma 7, datasource URLs are decoupled from declarative schema.prisma files and managed through a TypeScript configuration file (prisma.config.ts). This allows environment parameters to be resolved dynamically and type-safely at runtime and build time.
- **Driver Adapters** (@prisma/adapter-pg)**:** Prisma 7 replaces legacy internal binary query engines with JavaScript/TypeScript database driver adapters. Using @prisma/adapter-pg alongside the native Node.js pg connection pool (pg.Pool), queries execute through standardized connection pools optimized for serverless, edge, and containerized runtime environments.
- **Relational Schema Modeling (1:N):** Entities are modeled declaratively. A User (Author) connects to multiple Post records using @relation(fields: [authorId], references: [id], onDelete: Cascade). Cascading rules enforce referential integrity across related tables when parent records are modified or removed.
- **ACID Transactions** (prisma.$transaction)**:** Transactions guarantee that a sequence of database operations either completely succeed or completely roll back in the event of an error, preserving Atomicity, Consistency, Isolation, and Durability. Interactive transactions (prisma.$transaction(async (tx) => { ... })) ensure dependent multi-table writes execute inside an isolated lock context.
- **Global Singleton Connection Pattern:** Next.js Hot Module Replacement (HMR) re-evaluates server modules on every file change during development. Attaching PrismaClient to the Node.js globalThis object prevents the creation of redundant client instances and prevents exhausting the PostgreSQL connection pool.

```
                <form action={deletePostAction.bind(null,
post.id)}>
                  <button type="submit" style={{ background:
'#ef4444', color: '#fff', border: 'none', padding: '6px
12px', borderRadius: '4px', cursor: 'pointer' }}>
                    Delete
                  </button>
                </form>
              </div>
            ))
          )}
        </div>
      </div>
    );
}
```

Step 8: Validate Database & Visual Inspection

1. Run Prisma Studio in a dedicated terminal:

```
npx prisma studio
```

2. Start the development server:

```
npm run dev
```

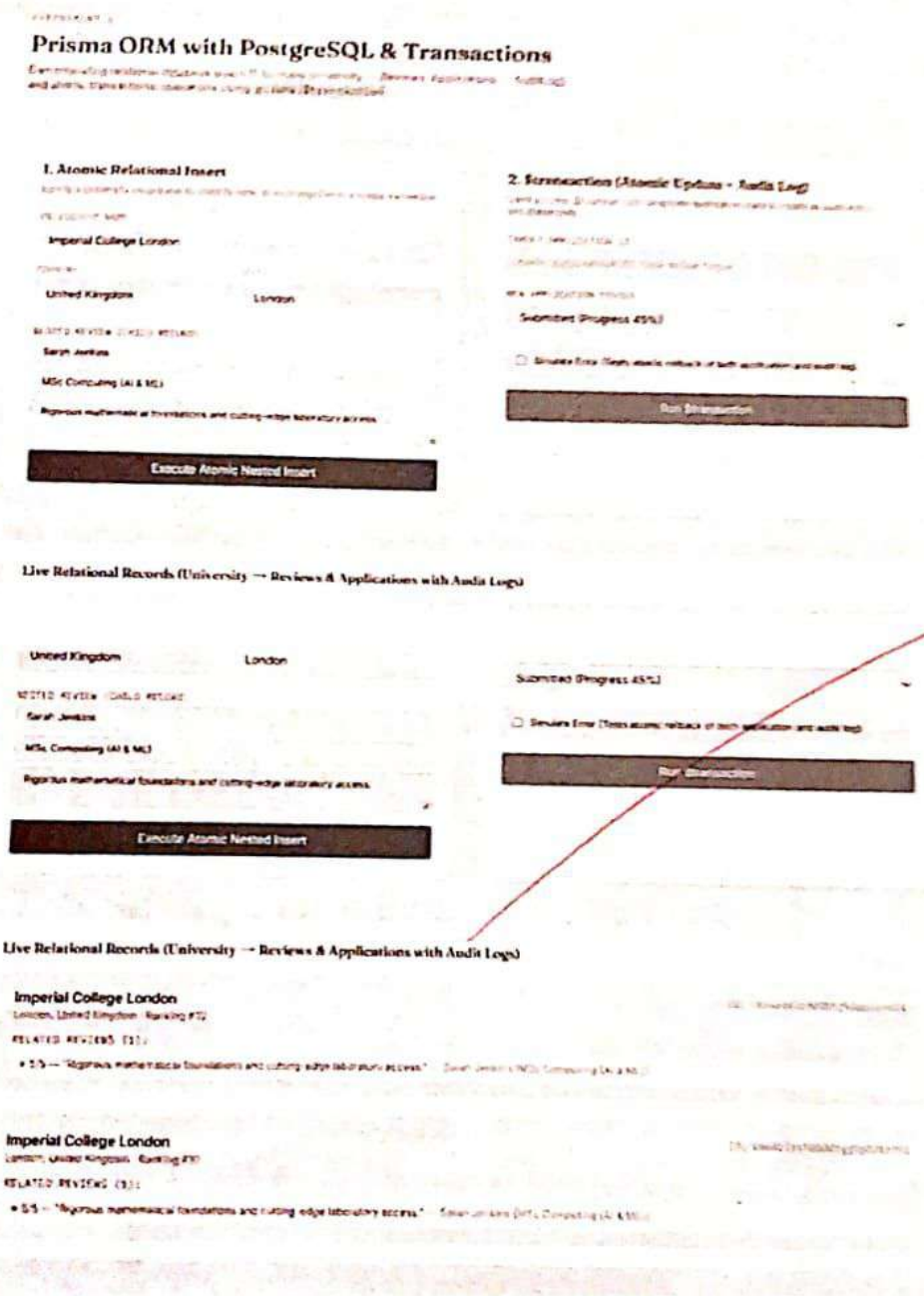
3. Navigate to http://localhost:3000/blog-db, submit a new record, and verify database writes in the web viewport and in Prisma Studio (http://localhost:5555).

OBSERVATION:

Observe how Prisma 7 integrates with the containerized PostgreSQL database using the @prisma/adapter-pg driver adapter.

When executing mutations, prisma.$transaction() initiates an atomic SQL transaction block (BEGIN -> User.upsert -> Post.create -> COMMIT). If an error occurs during either step, the database state rolls back completely (ROLLBACK), preventing orphan records and ensuring referential integrity across the relation.

A database layer using prisma ORM with PostgreSQL / MySQL to wire relationship logic & handle full transactional operations was implemented.



CONCLUSION:

A containerized PostgreSQL database layer was successfully established using Docker Compose and integrated with Prisma 7 in Next.js. The experiment verified programmatic configuration (prisma.config.ts), driver adapter integration (@prisma/adapter-pg), 1:N relational modeling, singleton client pooling, and atomic data mutations using interactive ACID transactions (prisma.$transaction).

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA 3.18)

Department of Artificial Intelligence and Machine Learning

B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

POST LAB QUESTION:

Q: Why does Prisma 7 require a driver adapter like @prisma/adapter-pg instead of using the built-in query engine for direct TCP connections?

A:

Prisma 7 decouples the ORM client from proprietary Rust-based internal network engines in favor of standard, modular JavaScript/TypeScript database driver adapters.

Connection Pool Re-use: It allows Prisma to leverage battle-tested connection pool implementations (like pg.Pool), reducing memory overhead and offering direct control over socket timeouts and max connection limits.

Serverless & Edge Compatibility: Driver adapters enable Prisma to execute across standard Node.js, Edge runtimes (Cloudflare Workers, Vercel Edge), and WebAssembly environments using unified driver interfaces without requiring OS-specific binaries.

Conclusion :

A containerized PostgreSQL database layer was successfully established using Docker compose & integrated with Prisma 7 in Next.js.